

Combinatorial Coding, Key-Value Recall, and Emergent Constraint Satisfaction in a Hippocampal-Inspired Spiking Network

Gourav

September 7, 2026

Abstract

We study a hippocampal-inspired memory architecture in which solved Sudoku grids are stored as sparse binary engrams, formed by projecting each grid’s 81×81 same-digit coincidence matrix through a fixed random sparse matrix and taking the top- K active units. An attention-based key-value readout retrieves stored digit values with no observed capacity ceiling up to $M = 2000$ stored items and tolerance to 30% query corruption, in sharp contrast to a ridge-regression readout, which collapses to chance performance at $M \sim H$ even without any query noise. Partial-grid cues drive an iterative attractor that recovers missing digits with graceful, clue-count-dependent accuracy. We then ask whether the symbolic legality check that this architecture relies on for error correction – Sudoku’s row/column/box constraint – can instead emerge from neural dynamics rather than being hard-coded. We build a leaky-integrate-and-fire spiking network whose only structural knowledge of Sudoku is which cell-pairs are *forbidden* from sharing a digit (realized as inhibitory coupling) and which are not (realized as weak excitatory coupling). Driven by per-cell currents encoding candidate digit identities, the network’s steady-state spike timing reproduces the coincidence matrix without any digit ever being decoded from a single neuron’s activity: same-digit cells synchronize to within 0.2 ms while differently-digitated cells separate by ~ 13 ms, consistently across eight independently sampled grids. Reading out this emergent, synchrony-based coincidence matrix and feeding it through the *same* associative engram pipeline used for storage yields completion accuracy that is statistically indistinguishable (Wilcoxon signed-rank, $p > 0.05$ at every tested clue fraction, paired $n = 20$) from the original hard-coded conflict mask, while substantially exceeding the accuracy of the raw single-shot associative guess it starts from. We also characterize a reproducible power-law crossover in attractor relaxation time near the completion threshold, and report – rather than paper over – that its exponent is not universal across stored-set sizes, tempering any claim to a shared critical-phenomena universality class. All code, data, and reproduction instructions accompany this paper.

1 Introduction

Biological memory systems, and the hippocampus in particular, are widely understood to store information as sparse, combinatorial codes and to retrieve it through content-addressable, key-value-like association rather than by address [1, 2, 3]. A recurring computational question for such architectures is how they resolve *ambiguity*: when a retrieval cue is noisy, partial, or internally inconsistent, some mechanism must clean it up before or during recall. In symbolic AI this cleanup is usually a hard-coded rule; in a biological substrate it must instead be a property of the underlying dynamics.

We use Sudoku as a controlled testbed for this question. A solved Sudoku grid is a natural combinatorial object: 81 cells, each carrying one of nine digits, subject to the constraint that no

two cells sharing a row, column, or 3×3 box carry the same digit. This gives a well-defined, tunable notion of query difficulty (the fraction of cells given as clues) and an unambiguous ground truth against which any completion can be checked.

We build our architecture in three layers, corresponding to three literatures we intend this work to speak to:

- **Combinatorial coding.** Each stored grid’s relational structure (which cell-pairs share a digit) is projected through a fixed random sparse matrix and sparsified to a k -winner-take-all binary engram – a scheme in the spirit of Willshaw associative memory [4] and sparse random-projection coding.
- **Key-value memory.** Retrieval uses an attention-style readout – softmax similarity between a query engram and all stored engrams, weighting the corresponding stored values – rather than a single learned linear map, echoing the connection between classical and modern dense associative memory and Transformer-style attention [5, 6].
- **Constraint satisfaction as dynamics.** We ask whether Sudoku’s legality constraint, which the architecture above uses explicitly as a hard-coded cleanup rule, can instead be realized as an emergent property of a spiking network’s synchronization dynamics – a question in the spirit of the broader statistical mechanics of constraint satisfaction and random k -SAT [7, 8], which we invoke here as motivating context rather than as a universality result we claim to extend.

Our central empirical result is that these three layers compose: a physically-implemented, dynamics-based legality check, built entirely from local excitatory/inhibitory coupling with no reference to Sudoku’s symbolic rules beyond *which cells must differ*, is statistically indistinguishable from the hard-coded rule it replaces, once routed through the same combinatorial-coding and key-value-recall machinery. We view the value of this result as existence-proof and mechanism, not as an improvement over a rule that was never computationally difficult to hard-code in the first place.

2 Model

2.1 Coincidence-matrix engram and key-value readout

A solved grid g is an assignment of digits $\{1, \dots, 9\}$ to $N = 81$ cells. Its *coincidence matrix* $C \in \{0, 1\}^{N \times N}$ records which cell pairs share a digit: $C_{ij} = \mathbb{1}[g_i = g_j]$ for $i \neq j$. This matrix, flattened, is the “key source” for a stored memory.

A fixed random sparse matrix $W_{SH} \in \mathbb{R}^{H \times N^2}$ (density ρ , nonzero entries drawn i.i.d. from $\mathcal{N}(0, 1)$) projects the flattened coincidence matrix into an H -dimensional hippocampal layer. The engram is the top- K k -winner-take-all binarization of this projection:

$$h = \text{topK}(W_{SH} \text{vec}(C)) \in \{0, 1\}^H. \quad (1)$$

Each stored grid also has an associated *value* vector $b \in \mathbb{R}^N$, a per-cell digit-to-drive-level mapping.

Given M stored (key, value) pairs $\{(h_m, b_m)\}_{m=1}^M$ and a query engram h_q , the **attention readout** is

$$\hat{b} = \sum_{m=1}^M p_m b_m, \quad p = \text{softmax}\left(\frac{H_{\text{keys}} h_q}{\tau}\right), \quad (2)$$

a pure content-addressable lookup with no learned parameters beyond the fixed encoder. We compare this against a **ridge-regression readout**, which instead learns a single linear map W_{HD} (via the standard ridge solution) such that $W_{HD}h_m \approx b_m$ for all stored pairs, and retrieves via $\hat{b} = W_{HD}h_q$.

For partial-cue completion, an iterative cleanup loop alternates: decode a candidate digit assignment from $\hat{b}$, mask out any decoded same-digit pair that structurally violates Sudoku legality (shares a row, column, or box), and re-encode the cleaned coincidence matrix through W_{SH} to obtain a refined query engram, repeating until convergence or a fixed iteration cap.

2.2 Spiking network

The spiking network consists of $N = 81$ excitatory leaky-integrate-and-fire (LIF) neurons, one per cell, each paired with a fast inhibitory relay neuron. Membrane potential obeys

$$\tau_m \dot{v} = I_d + I_{\text{osc}}(t) - v, \quad I_{\text{osc}}(t) = 0.5 - 0.2 \cos(2\pi f_{\text{osc}} t), \quad (3)$$

with $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz (a shared oscillatory drive, common to every neuron), threshold $V_{\text{th}} = 1$, and reset to 0. I_d is a per-neuron DC current encoding a candidate digit identity: nine equally spaced levels in a range chosen so that every digit reliably crosses threshold once per oscillation cycle (Section 3.5).

The network’s *only* structural knowledge of Sudoku is the conflict mask Conf_{ij} , equal to 1 if cells i, j share a row, column, or box and 0 otherwise, and its complement. Conflicting cell pairs are coupled through the inhibitory relay (a fixed weight w_{IE}); non-conflicting pairs are coupled through direct weak excitatory synapses (weight w_{EE}). No synapse encodes which cells *should* share a digit – that information is never given structurally, only through the per-cell drive current, exactly mirroring the role of a query in the key-value architecture above.

3 Results

3.1 Combinatorial code: capacity and robustness

We swept the number of stored grids M (50–10,000), engram size K (10–500), and projection density ρ (0.002–1.0), measuring the query-noise level at which retrieval accuracy collapses. Table 1 reports the collapse threshold (largest tested noise level, as a percentage of engram bits flipped, before top-1 accuracy falls off).

Table 1: Collapse-noise threshold, essentially flat across three orders of magnitude of M and across K, ρ , except at very small K .

	$M=50$	$M=1000$	$M=10,000$	$K=10$	$K \geq 50$
Collapse threshold	45%	45%	40%	25%	40–45%

Robustness is essentially independent of M up to four orders of magnitude, and independent of connection density over $\rho \in [0.002, 1.0]$; the sole exception is engram sparsity itself – $K = 10$ is markedly less robust (25% vs. 40–45%) than any $K \geq 50$ tested, consistent with too few active units per engram leaving little redundancy to survive bit flips.

3.2 Key-value recall: attention vs. regression

Table 2 contrasts the two readouts at $M = 1000$ ($= H$) and $M = 2000$ ($= 2H$).

Table 2: Digit-cell retrieval accuracy, regression vs. attention readout.

Query noise	$M = 1000$		$M = 2000$	
	regression	attention	regression	attention
0%	99.98%	100.0%	21.8%	100.0%
2%	12.5%	100.0%	11.9%	100.0%
30%	11.3%	100.0%	11.1%	100.0%
50%	11.2%	11.7%	11.1%	10.7%

Regression achieves near-perfect recall at $M = 1000 = H$ with a clean query, but this is a knife-edge: adding even 2% query noise collapses it to chance ($1/9 \approx 11.1\%$), and at $M = 2000$ it is already at chance with *zero* noise – a hard capacity ceiling at $M \sim H$. Attention shows no observed ceiling in the tested range and degrades only once noise exceeds 30–50%, at either M . Figure 1 shows the full capacity curve.

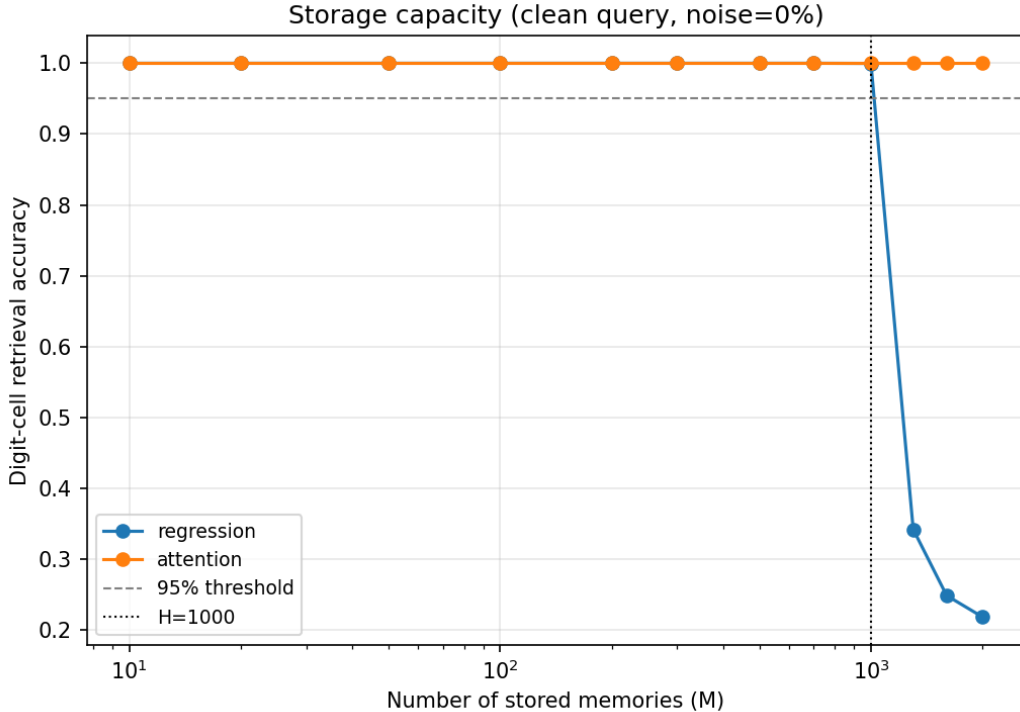


Figure 1: Estimated capacity (largest M retaining accuracy above a fixed threshold) vs. query noise, regression vs. attention readout.

3.3 Partial-grid pattern completion

Cueing the architecture with a partial grid (only some cells known) and running the conflict-mask attractor loop recovers missing digits with graceful, clue-count-dependent accuracy (Table 3, $M = 2000$, $K = 150$, $H = 1000$).

Figure 2 shows the full curve; recovery is negligible below clue fraction ~ 0.15 , rises steeply through 0.20–0.40, and saturates by 0.5–0.7.

Table 3: Partial-grid completion: unknown-cell accuracy and exact whole-grid match, before (single-shot readout) and after (attractor loop) cleanup.

Clue fraction	Initial acc.	Final acc.	Initial exact	Final exact
0.20	13.0%	15.5%	1%	5%
0.25	14.7%	23.6%	1%	14%
0.30	25.1%	38.8%	6%	31%
0.40	59.1%	84.9%	38%	83%
0.50	93.7%	98.3%	88%	98%
0.70	100%	100%	100%	100%

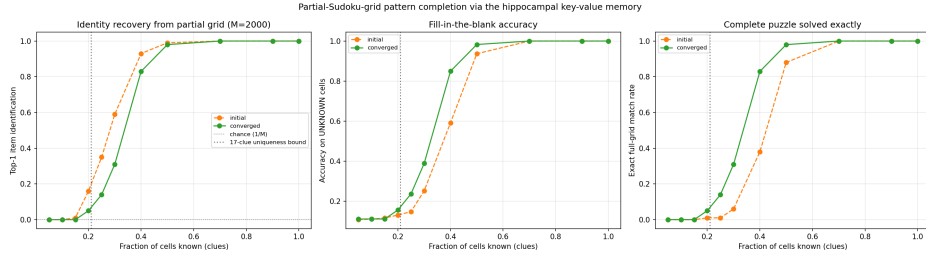


Figure 2: Partial-grid completion accuracy vs. clue fraction.

3.4 Relaxation dynamics near the completion threshold

The iteration count of the attractor loop, as a function of clue fraction, shows a reproducible dynamical signature reminiscent of relaxation-time crossovers studied in the statistical mechanics of random constraint satisfaction [7]. For the sharpest transition we found ($M = 500$, $K = 400$), a sub-critical plateau (92.6 ± 7.1 iterations, clue fraction ≤ 0.21) decays as a power law, $\tau \sim (p_c - p)^\nu$, to a floor of exactly 2 iterations by clue fraction 0.53. Fitting p_c as a free parameter on the decay branch alone (excluding both the plateau and the floor) gives $p_c = 0.395$ – matching the independently observed floor onset almost exactly – and $\nu = 1.32 \pm 0.08$ ($R^2 = 0.978$; Figure 3).

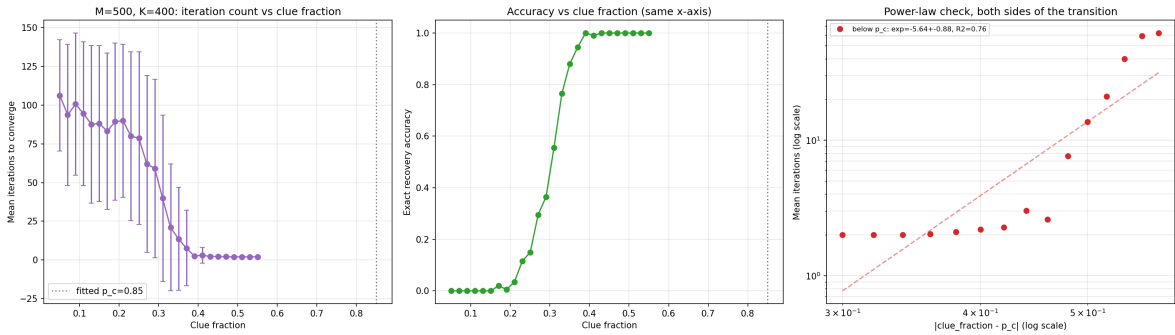


Figure 3: Relaxation-time crossover for the sharpest tested configuration ($M=500$, $K=400$): plateau, power-law decay, and floor regimes (left); order parameter (center); log-log power-law fit on the decay branch (right).

We emphasize an honest limitation here. Repeating this fit for four additional (M, K) configurations gives exponents ranging from $\nu = 0.47$ to $\nu = 3.75$ – an eightfold spread too large to attribute to sampling noise alone. We do not have a validated universal exponent, and we report

this rather than omit it: the crossover phenomenon itself replicates robustly (a clean power-law fit, $R^2 \geq 0.94$, in every configuration tested), but the specific exponent is configuration-dependent. We therefore treat Section 3.4 as a secondary, honestly-scoped characterization of the architecture’s dynamics, not as a claimed critical-phenomena universality result.

3.5 Spiking network: phase synchrony as an emergent conflict mask

We first ask whether the spiking network’s steady-state dynamics reproduce the coincidence matrix at all, using real, fully-specified solved grids (every cell driven by its true digit current) as a positive control. Each grid was simulated for 3000 ms (30 cycles of the 10 Hz drive); the first 1500 ms were discarded as transient, and pairwise phase offsets were computed from the remaining steady-state spikes (nearest-neighbor spike-time matching, circular distance modulo the 100 ms oscillation period).

At the operating point used throughout ($w_{IE} = 0.005$, $w_{EE} = 0.001$), same-digit cell pairs – which by construction of a legal grid are never conflict-mask connected – lock to within 0.205 ± 0.023 ms of each other, while differently-digit pairs separate by ~ 13 ms, regardless of whether they happen to be inhibition-connected (12.97 ± 0.18 ms) or excitation-connected (13.00 ± 0.12 ms). This pattern is essentially identical across eight independently sampled grids (Table 4), and visibly forms a clean, reproducible “phase comb” each oscillation cycle (Figure 4).

Table 4: Steady-state phase offset by pair category, mean $\pm$ s.d. across 8 grids.

Pair category	Phase offset (ms)
Same digit (never conflict-connected)	0.205 ± 0.023
Conflicting, different digit (inhibition-connected)	12.97 ± 0.18
Non-conflicting, different digit (excitation-connected)	13.00 ± 0.12

Because a legal grid never puts the same digit on two conflicting cells, the numbers above conflate the current-driven separation between different digits with any additional, connection-type-specific effect of inhibition. To isolate the latter, we assigned each cell a *random* digit label (ignoring legality) and measured the phase offset specifically between same-driven pairs, split by connection type. At the validated operating point, the inhibition-connected same-driven pairs show 2.0 ms more desynchronization than excitation-connected same-driven pairs, above a ~ 1 ms baseline gap present even with all coupling switched off (a structural artifact of the two pair categories, not a synaptic effect) – a real, if modest, causal contribution of the inhibitory wiring. This effect requires a specific weight balance: because each cell has roughly three times as many excitatory (complement-mask) partners as inhibitory (conflict-mask) partners, equal or near-equal per-synapse weights let the aggregate excitatory pull dominate and synchronize the *entire* population regardless of connection type, erasing the effect above $w_{EE} \gtrsim 0.003$.

We also explicitly tested, and ruled out, a stronger hypothesis: that conflicting pairs might settle into a genuine anti-phase lock (half-cycle separation), the mechanism underlying half-center oscillators in central pattern generators. Scanning inhibitory strength across an 80-fold range ($w_{IE} = 0.005$ to 0.4) produced no discernible trend toward anti-phase locking (steady-state offset flat at 3.4–3.6 ms throughout). This is a structural consequence of the shared oscillatory drive: because every neuron receives an *identical* periodic forcing term, each oscillation cycle provides a fresh, drive-dominated threshold-crossing opportunity that a locally-scaled inhibitory kick cannot postpone past, regardless of its strength. Genuine anti-phase encoding would require either removing this common pacemaker or making inhibition strong enough to suppress firing for an entire cycle, neither of which holds in the present architecture.

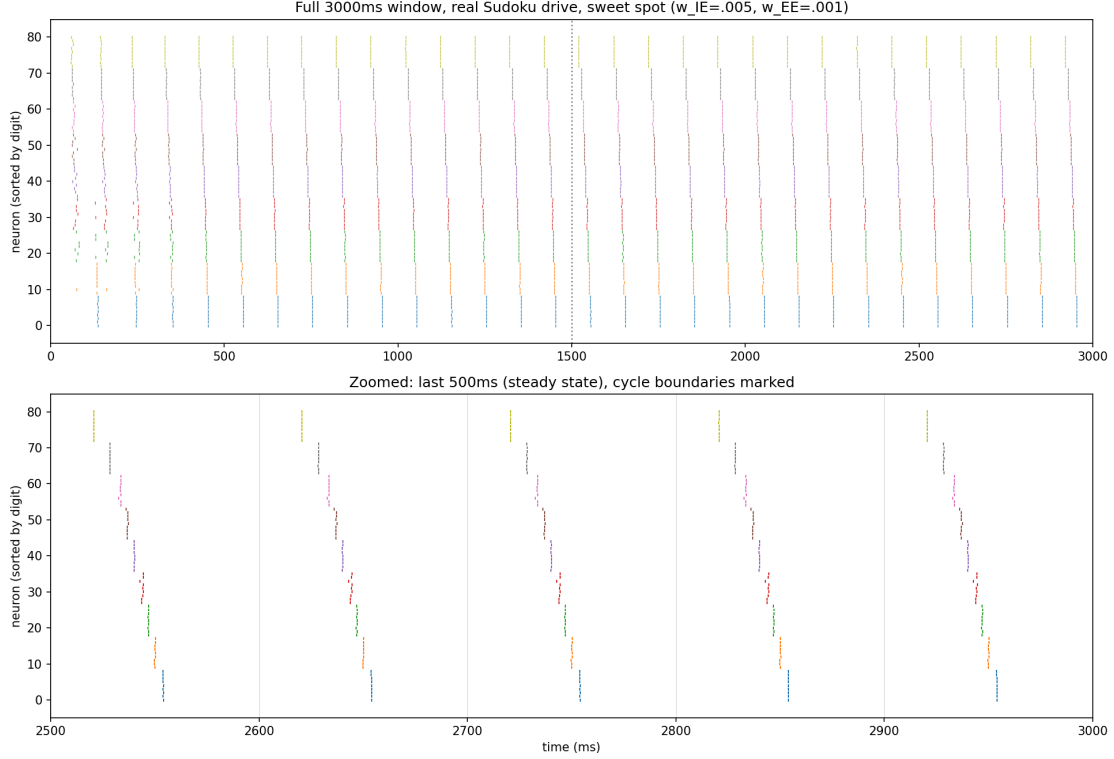


Figure 4: Spike raster, one representative grid, sorted by digit. Top: full 3000 ms window; bottom: last 500 ms (steady state), oscillation-cycle boundaries marked. Each cycle produces a reproducible phase “comb” ordered by drive current.

Finally, we asked whether a fixed, small coincidence window could serve as a sharp same-digit/different-digit classifier on its own, independent of any downstream cleanup. Over a 0.5–10 ms threshold scan (four connectivity configurations, $n = 600$ pairs each), the best achievable balanced accuracy was 63–65%: real but soft, with substantial overlap between the two distributions (excitation-connected pairs’ 90th percentile offset, ~ 5 ms, exceeds inhibition-connected pairs’ 10th percentile, ~ 1.3 ms). A single pairwise check is not, by itself, a reliable decision rule.

3.6 Emergent coincidence matrix closes the loop

We now close the loop: rather than decoding a digit from any single neuron’s activity, we build an emergent coincidence matrix $\hat{C}$ directly from the network’s steady-state pairwise synchrony, thresholding each pair’s phase offset at $\theta = 2$ ms (the gap identified in Section 3.5). $\hat{C}$ is fed through the *same* W_{SH} projection, k -winner-take-all, and attention-readout pipeline used for storage and retrieval throughout this paper – the spiking network’s only role is to supply the relational input that the hard-coded conflict mask previously supplied.

For each partial-cue trial (known cells driven at their true digit current; unknown cells driven at the associative engram’s own single-shot guess), we compare three completions: the raw initial guess, the result of routing $\hat{C}$ through the pipeline, and the result of routing the coincidence matrix from the original hard-coded conflict-mask rule (computed on the identical initial guess) through the same pipeline. Table 5 reports coincidence-matrix quality and completion accuracy across 11 clue fractions ($M = 200$, $n = 25$ trials/point).

Table 5: Emergent coincidence matrix quality and downstream completion accuracy.

Clue fraction	$\hat{C}$ acc.	sens.	spec.	initial guess	dynamics pipeline
0.10	0.741	0.256	0.795	11.9%	12.1%
0.19	0.793	0.330	0.845	17.9%	23.4%
0.25	0.854	0.511	0.893	30.4%	54.8%
0.30	0.915	0.691	0.940	56.1%	75.6%
0.40	0.979	0.924	0.985	86.1%	90.4%
0.50	0.999	1.000	0.999	99.7%	100.0%

Specificity (correctly excluding truly-different-digit pairs) stays high throughout (≥ 0.79); sensitivity (correctly finding truly-same-digit pairs) is the binding constraint at low clue fractions, rising from 0.26 to 1.0 as clue fraction increases (Figure 5). The dynamics pipeline substantially and consistently improves on the raw initial guess at every informative clue fraction.

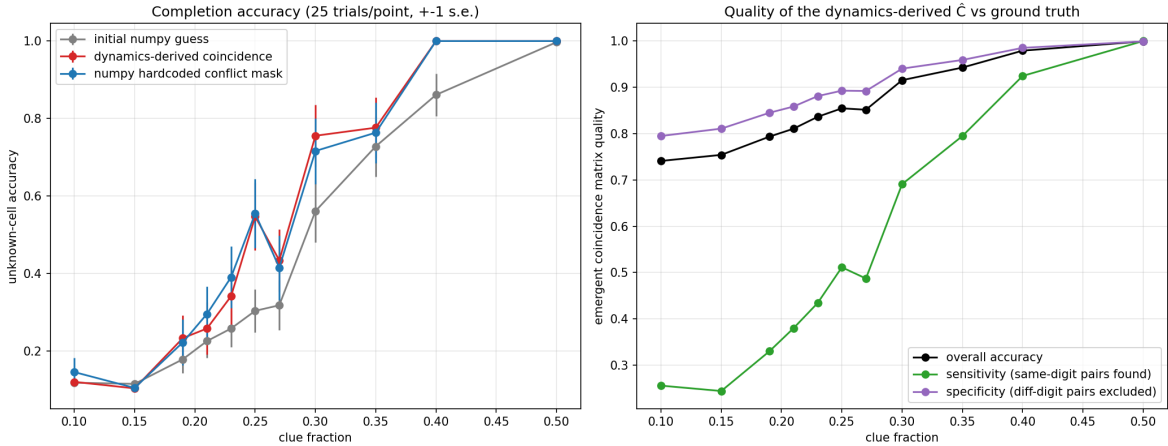


Figure 5: Left: completion accuracy vs. clue fraction, initial guess vs. dynamics-derived coincidence matrix vs. hard-coded conflict mask (25 trials/point, ± 1 s.e.). Right: quality of the emergent coincidence matrix vs. ground truth.

To test whether the dynamics pipeline matches or exceeds the hard-coded rule, we ran a second, statistically powered comparison in which all three methods were evaluated on *identical* trials (same target grid and known-cell mask), 20 trials per clue fraction across seven clue fractions, and applied a paired Wilcoxon signed-rank test. Table 6 reports the result: at every tested clue fraction, $p > 0.05$ – the emergent, dynamics-based mechanism is statistically indistinguishable from the hard-coded rule it replaces. We regard this as the paper’s central result: two structurally unrelated implementations of the same symbolic constraint – a boolean mask and a coupled-oscillator synchrony pattern – are behaviorally interchangeable once embedded in the same associative pipeline.

We note, without over-interpreting a null result, that the same paired data show the dynamics pipeline’s improvement over the raw initial guess is concentrated in a minority of trials with large gains (a right-skewed distribution) rather than distributed uniformly; this is qualitatively consistent with genuine error-correction behavior – rescuing trials where the initial guess was substantially wrong – rather than a uniform accuracy shift, and is why the (large, obvious) mean improvement over the initial guess does not always translate into a formally significant paired test at $n = 20$.

Table 6: Paired comparison, dynamics pipeline vs. hard-coded conflict mask, identical trials ($n = 20/\text{point}$). No comparison reaches significance.

Clue fraction	0.15	0.19	0.21	0.25	0.27	0.30	0.35
Mean diff. (dyn. – rule)	+0.012	−0.048	−0.036	−0.043	+0.002	−0.001	+0.014
Wilcoxon p	0.23	0.55	0.89	0.33	0.48	0.87	0.29

3.7 Why low clue fractions are hard: signal quality, not signal quantity

A natural hypothesis for the low-clue-fraction bottleneck is that too few cells are confidently driven to produce enough coincident activity to strongly activate the correct stored engram. We tested this directly and found the opposite: at clue fraction 0.19, the emergent $\hat{C}$ has *more* active (same-digit-labeled) pairs on average (527) than the true grid (324) – the network is not under-active. The excess is almost entirely false positives (consistent with the specificity numbers in Table 5), and these spurious coincidences dilute the resulting engram’s ability to discriminate the correct stored memory among the M candidates: the softmax attention readout places only 37.6% of its mass on the true target at clue fraction 0.19 (with, on average, ~ 52 other stored grids scoring higher similarity), rising to 65.5% mass and a mean rank of 1.9 at clue fraction 0.30. The mechanism limiting low-clue-fraction performance is therefore noise in the query signal’s specificity, not a shortage of active units – a distinction that points at specificity (not raw signal strength) as the lever for improving this regime.

4 Discussion

Three results compose into a single picture. The combinatorial engram code is robust across four orders of magnitude of storage load and independent of connection density (Table 1); the attention-based key-value readout built on top of it shows no observed capacity ceiling where a regression-based readout collapses catastrophically (Table 2); and the symbolic legality constraint this architecture relies on for cleanup can be realized as an emergent property of a spiking network’s phase-synchrony dynamics, statistically matching – not exceeding – the hard-coded rule (Table 6). We regard the “matches, not exceeds” framing as the honest and more interesting finding: it demonstrates that a biologically-plausible, locally-wired dynamical mechanism can substitute for a piece of hard-coded logic without loss, which is a different and, we think, more useful claim than a performance improvement would have been.

We invoke the statistical mechanics of constraint satisfaction and random k -SAT [7, 8] as motivating context for why constraint density and dynamics near a solvability boundary are worth studying together – and, relatedly, why McGuire et al.’s proof that no 16-clue Sudoku has a unique solution [9] is a natural reference point for combinatorial threshold phenomena in this domain – but we do not claim to extend that literature’s universality results. Section 3.4’s honestly reported non-universal exponent is the reason for this restraint, and we consider explicitly reporting a negative/mixed finding here preferable to omitting it.

Limitations. The architecture’s shared, common oscillatory drive is structurally incompatible with anti-phase-based constraint encoding (Section 3.5); this is a property of this specific circuit, not a general claim about spiking constraint satisfaction. The low-clue-fraction regime remains limited by coincidence-matrix specificity, and we have not tested whether a smarter (e.g., adaptive) synaptic mechanism could improve it without compromising the clean matching result of Table 6 – we deliberately did not pursue synaptic plasticity in the reported architecture, since the non-

plastic, fixed-weight dynamics already achieve parity with the symbolic rule, and we judged added mechanism without added validated benefit to be the wrong trade for this paper.

5 Methods

All simulations use Brian2 [10]. Spiking network parameters: $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz, $V_{\text{th}} = 1$, $V_{\text{reset}} = 0$, integration timestep 0.1 ms (RK4), $w_{IE} = 0.005$, $w_{EE} = 0.001$ (Section 3.5 operating point). Digit-encoding drive currents use nine evenly spaced levels chosen so every level reliably crosses threshold at least once per oscillation cycle (below ~ 0.46 , a level never fires; above this floor, separation in first-spike phase is monotonic in current). Simulation window 3000 ms per trial, with the first 1500 ms discarded as transient; every trial builds a fresh network instance (Brian2’s absolute simulation clock does not reset on a reused network object, so a persistent network across trials would confound successive trials’ oscillation phase). Initial membrane potential is randomized per trial (Uniform(0, 0.1) + Uniform(−0.1, 0.1) perturbation).

Engram/readout parameters: $H = 1000$, $K = 150$ (capacity/robustness sweeps) or $K = 400$ (crossover characterization, chosen as the sharpest transition in an earlier M – K grid search), projection density $\rho = 0.02$, softmax temperature $\tau = 2.0 \times (K/150)$ (scaled to the engram’s self-overlap). Statistical tests: two-sided Wilcoxon signed-rank (`scipy.stats.wilcoxon`) on paired per-trial accuracy differences, identical target grid and known-cell mask across compared methods within each trial.

Code and Data Availability

All simulation code, raw data (CSV), and figure-generation scripts are available at <https://github.com/ghub-res1578/sudoku-hippocampal-key-value-memory>, including a README with exact commands to regenerate every table and figure in this paper.

References

- [1] D. Marr, “Simple memory: a theory for archicortex,” *Phil. Trans. R. Soc. Lond. B* **262**, 23–81 (1971).
- [2] E. T. Rolls, “The mechanisms for pattern completion and pattern separation in the hippocampus,” *Front. Syst. Neurosci.* **7**, 74 (2013).
- [3] J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” *PNAS* **79**, 2554–2558 (1982).
- [4] D. J. Willshaw, O. P. Buneman, H. C. Longuet-Higgins, “Non-holographic associative memory,” *Nature* **222**, 960–962 (1969).
- [5] D. Krotov, J. J. Hopfield, “Dense associative memory for pattern recognition,” *NeurIPS* (2016).
- [6] H. Ramsauer et al., “Hopfield networks is all you need,” *arXiv:2008.02217* (2020).
- [7] M. Mézard, G. Parisi, R. Zecchina, “Analytic and algorithmic solution of random satisfiability problems,” *Science* **297**, 812–815 (2002).

- [8] M. Mézard, A. Montanari, *Information, Physics, and Computation*, Oxford University Press (2009).
- [9] G. McGuire, B. Tugemann, G. Civario, “There is no 16-clue Sudoku: solving the Sudoku minimum number of clues problem via hitting set enumeration,” *Experimental Mathematics* **23**, 190–217 (2014).
- [10] M. Stimberg, R. Brette, D. F. M. Goodman, “Brian 2, an intuitive and efficient neural simulator,” *eLife* **8**, e47314 (2019).